

Deep Learning & Neural Networks

A Structured Technical Overview

1. Introduction

Deep learning is a subfield of machine learning that uses artificial neural networks with many layers to model complex patterns in data. Inspired by the structure of the human brain, deep learning has enabled breakthroughs in image recognition, natural language processing, speech synthesis, and autonomous decision-making. Unlike classical machine learning methods that require hand-crafted features, deep learning algorithms learn hierarchical representations automatically from raw data.

The renaissance of deep learning began in 2012 when AlexNet won the ImageNet competition by a large margin using convolutional neural networks trained on GPUs. Since then, the field has grown exponentially, driven by the availability of large datasets, powerful hardware accelerators, and open-source frameworks such as TensorFlow and PyTorch.

2. Fundamentals of Neural Networks

2.1 Neurons and Layers

An artificial neuron receives one or more inputs, computes a weighted sum, adds a bias term, and passes the result through an activation function to produce an output. When many neurons are arranged in layers and connected to one another, they form a neural network. The input layer receives raw features, hidden layers learn intermediate representations, and the output layer produces the final prediction.

The depth of a network — the number of hidden layers — determines its capacity to represent complex functions. Shallow networks with one hidden layer are universal function approximators in theory, but deep networks require exponentially fewer neurons to represent the same functions in practice, making depth a key advantage.

2.2 Activation Functions

Activation functions introduce non-linearity into the network, enabling it to learn complex mappings. Without non-linearity, stacking multiple layers would be equivalent to a single linear transformation. Common activation functions include:

ReLU (Rectified Linear Unit): Returns $\max(0, x)$. Simple, computationally efficient, and avoids the vanishing gradient problem. The most widely used activation in deep networks.

Sigmoid: Maps any input to a value between 0 and 1. Historically popular but suffers from vanishing gradients in deep networks. Still used in output layers for binary classification tasks.

Softmax: Converts a vector of raw scores into a probability distribution. Used in the output layer for multi-class classification problems.

3. Training Neural Networks

3.1 Backpropagation

Backpropagation is the algorithm used to compute gradients of the loss function with respect to the network weights. It applies the chain rule of calculus to propagate error signals from the output layer back through the hidden layers. These gradients are then used by an optimisation algorithm to update the weights in the direction that reduces the loss.

The vanishing gradient problem occurs when gradients become exponentially small as they propagate through many layers, slowing or halting learning in early layers. Techniques such as ReLU activations, batch normalisation, residual connections, and careful weight initialisation help mitigate this problem.

3.2 Optimisation Algorithms

Stochastic Gradient Descent (SGD) updates weights using the gradient computed on a small random mini-batch of training examples. Mini-batch SGD provides a balance between the stability of full-batch gradient descent and the speed of online learning. Momentum accelerates SGD by accumulating a velocity vector in the direction of consistent gradients, helping the optimiser navigate flat regions and narrow valleys.

Adam (Adaptive Moment Estimation) combines momentum with adaptive learning rates for each parameter. It maintains estimates of both the first moment (mean) and second moment (uncentred variance) of the gradients and uses these to scale the learning rate per parameter. Adam is the default choice for most deep learning tasks due to its fast convergence and robustness to hyperparameter choices.

3.3 Regularisation

Overfitting occurs when a model performs well on training data but poorly on unseen data. Regularisation techniques reduce overfitting by constraining model complexity. L2 regularisation (weight decay) adds a penalty proportional to the square of the weights to the

loss function. Dropout randomly deactivates a fraction of neurons during each training step, forcing the network to learn redundant representations and reducing co-adaptation between neurons.

Data augmentation artificially increases the size and diversity of the training set by applying random transformations such as flipping, cropping, rotation, and colour jittering to training images. Early stopping monitors validation loss and halts training when it begins to increase, preventing the model from memorising noise.

4. Convolutional Neural Networks

4.1 Architecture

Convolutional Neural Networks (CNNs) are specialised for processing grid-structured data such as images. They exploit three key ideas: local connectivity, weight sharing, and pooling. Convolutional layers apply a set of learnable filters across the spatial dimensions of the input, each filter detecting a specific local pattern such as an edge, texture, or colour gradient.

Pooling layers reduce spatial resolution by aggregating nearby feature values. Max pooling selects the maximum value in each pooling window, providing translation invariance and reducing computational cost. Fully connected layers at the end of the network combine learned features to produce the final class scores.

4.2 Landmark Architectures

AlexNet (2012) demonstrated that deep CNNs trained on GPUs could dramatically outperform hand-crafted methods on large-scale image recognition. VGGNet showed that depth using very small (3x3) convolutional filters is a critical component of strong performance. ResNet introduced residual connections that allow gradients to bypass layers, enabling the training of networks with hundreds of layers without degradation.

EfficientNet uses neural architecture search to find optimal network depth, width, and resolution scaling, achieving state-of-the-art accuracy with significantly fewer parameters than previous architectures. Vision Transformers (ViT) apply the Transformer architecture directly to sequences of image patches, rivalling CNNs on large datasets.

5. Recurrent Networks and Transformers

5.1 Recurrent Neural Networks

Recurrent Neural Networks (RNNs) process sequential data by maintaining a hidden state that is updated at each timestep. This hidden state acts as a memory of past inputs, enabling the network to model temporal dependencies. RNNs are used for tasks such as language modelling, speech recognition, and time-series forecasting.

Long Short-Term Memory (LSTM) networks address the vanishing gradient problem in vanilla RNNs through gating mechanisms. The forget gate controls which information to discard from the cell state. The input gate controls what new information to store. The output gate controls what information to expose as the hidden state. GRUs are a simplified variant with fewer parameters that often perform comparably to LSTMs.

5.2 The Transformer Architecture

The Transformer, introduced in the paper 'Attention Is All You Need' in 2017, replaced recurrence with self-attention mechanisms. Self-attention allows every position in a sequence to attend to every other position, capturing long-range dependencies without the sequential computation bottleneck of RNNs. Transformers are highly parallelisable and have become the dominant architecture for natural language processing.

BERT (Bidirectional Encoder Representations from Transformers) pre-trains a deep Transformer encoder on masked language modelling and next sentence prediction. Fine-tuning BERT on downstream tasks such as question answering and sentiment analysis achieves state-of-the-art results with relatively little task-specific data. GPT models use a Transformer decoder with autoregressive language modelling, generating text one token at a time conditioned on all previous tokens.

6. Applications

6.1 Computer Vision

Deep learning has transformed computer vision. Object detection models such as YOLO and Faster R-CNN locate and classify multiple objects within a single image in real time. Semantic segmentation assigns a class label to every pixel, enabling applications such as medical image analysis and autonomous driving scene understanding.

Generative Adversarial Networks (GANs) learn to generate realistic images by training a generator and discriminator in competition. StyleGAN can synthesise photorealistic human faces that do not belong to any real person. Diffusion models have recently surpassed GANs in image quality and diversity, powering tools like DALL-E and Stable Diffusion for text-to-image generation.

6.2 Natural Language Processing

Large language models (LLMs) pre-trained on internet-scale text corpora demonstrate remarkable capabilities in text generation, translation, summarisation, question answering, and code generation. GPT-4, PaLM, and similar models can follow complex instructions, reason across multiple steps, and adapt to new tasks with few examples.

Retrieval-Augmented Generation (RAG) enhances language models by retrieving relevant documents from a knowledge base and conditioning generation on the retrieved context. This allows models to access up-to-date information beyond their training data and reduces hallucination by grounding responses in retrieved evidence.

7. Challenges and Future Directions

7.1 Interpretability

Deep neural networks are often described as black boxes because their internal representations are difficult to interpret. This opacity is a significant barrier to deployment in safety-critical applications such as medical diagnosis, autonomous vehicles, and legal decision-making. Explainable AI (XAI) research develops methods to make model decisions transparent and understandable to human users.

Saliency maps highlight which parts of an input image most influenced a model's prediction. Concept Activation Vectors (CAVs) probe which human-defined concepts a layer has learned. Mechanistic interpretability seeks to reverse-engineer the algorithms implemented by neural network weights, identifying circuits responsible for specific behaviours.

7.2 Robustness and Safety

Adversarial examples are inputs crafted by adding imperceptible perturbations to fool a neural network into making incorrect predictions with high confidence. Adversarial attacks pose risks to autonomous vehicles, biometric authentication, and malware detection. Adversarial training, certified defences, and randomised smoothing are approaches to improving model robustness.

AI safety research addresses the challenge of ensuring that increasingly capable AI systems behave in accordance with human values and intentions. Reinforcement learning from human feedback (RLHF) trains models to follow human preferences. Constitutional AI embeds a set of principles into the training process to guide model behaviour toward being helpful, harmless, and honest.

7.3 Efficiency and Sustainability

Training large deep learning models requires enormous computational resources and energy. A single training run of a large language model can consume as much energy as hundreds of households over a year. Model compression techniques such as pruning, quantisation, and knowledge distillation reduce model size and inference cost without significant loss in accuracy.

Neural architecture search (NAS) automates the design of efficient network architectures. Hardware-aware NAS optimises models for specific deployment targets such as mobile devices and edge processors. Green AI is an emerging movement that advocates measuring and reducing the environmental footprint of machine learning research and deployment.